Ministerul Educației și Cercetării al Republicii Moldova
Universitatea Tehnică a Moldovei
Facultatea Calculatoare, Informatică și Microelectronică

Raport
la
Lucrarea de laborator Nr. 11

Disciplina: Tehnici de programare

Tema: ”Obiecte și clase.Windows în Python:inventar de procese și jurnalul de evenimente ”

A efectuat : st. gr.SI –263 Boicu Ecaterina

A verificat : asistent universitar Reițman Patricia

Chișinău – 2026

I.

SARCINA LUCRĂRII

În această lucrare modelez în Python procese și evenimente de pe o mașină Windows ca obiecte , nu ca dicționare separate.Folosesc clase declarate cu @dataclass, metode care răspund singure (e_suspect),o clasă-container (Inventar) care se umple din JSON și un raport de alarme comandat din terminal cu argparse.

Lucrarea are două părți:

LUCRAREA A (nucleu):clasele Proces și Eveniment, metoda e_suspect ,încărcarea din JSON.

LUCRAREA B (extinderea): clasa Inventar cu metode de interogare, raportul JSON/text și testele cu pytest.

STRUCTURA ŞI FIŞELE DE START:

Lucrarea se află în arsenal/windows/:

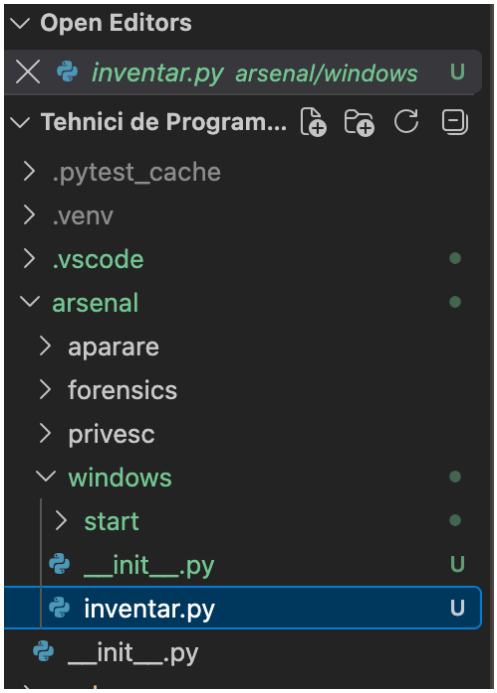


Fig.1 - Structura lucrării

Fișierele de start sunt două capturi de pe o mașină Windows de laborator :

Fișier	Conținut
procese.json	Lista proceselor : pid,nume,user,cale (7 procese)
evenimente.json	Jurnalul de securitate : id,nivel,sursa,cont,mesaj (8 evenimente)

Fig.1.2 - Fișiere JSON (tabel informativ)



Fig.1.3- Evenimente JSON



Fig.1.4- Procese JSON

II.

LUCRAREA A.

Procesul și evenimentul ca obiecte

A1. Clasa Proces , un tipar de date

Un proces are patru câmpuri legate : pid,nume,user,cale.Îl descriu ca @dataclass , cu numele și tipul fiecărui câmp:

```
p = Proces(3120, 'update.exe', 'ana', r'C:\Users\ana\AppData\Local\Temp\update.exe')
print(p.pid, p.nume)
print(p)
print(p.e_suspect())
"
3120 update.exe
Proces(pid=3120, nume='update.exe', user='ana', cale='C:\\Users\\ana\\AppData\\Local\\Temp\\update.exe')
True
```

Fig.2.1 - Rezultatul A1

Decoratorul @dataclass generează automat metodele __init__,__repr__ și __eq__. De aceea : Proces(3120,…) funcționează fără să scriu constructorul , iar print(p) afișează toate câmpurile , nu o adresă de memorie.
Câmpurile se citesc cu punct (p.pid) , nu cu paranteze pătrate (p[“pid”]) ca la dicționar.
Calea e scrisă cu r”…” (șir brut), ca “ \ “ Windows să rămână “ \ “ . În print(p) apare dublat “ \\”, pentru că așa afișează Python un backslash în interiorul unui șir.

A2. Metoda e_suspect pe Proces

Un proces care rulează din folderul utilizatorului (Temp Roaming) este un semn clasic de malware. Pun întrebarea ca metodă: obiectul se uită la propria cale și răspunde.



Fig.2.2 - Rezultatul A2

A3.Clasa Eveniment și ID-urile Windows

Evenimentele Windows au ID-uri cu sens:

ID	Semnificație
4625	Logon eșuat
1102	Jurnalul de audit a fost șters
7045	Serviciu nou instalat

Fig.2.3- Tabelul explicativ a clasei Eveniment

Eveniment este tot un @dataclass , cu propria metodă e_suspect

```
@dataclass
class Eveniment:
    id: int
    nivel: str
    sursa: str
    cont: str
    mesaj: str

    def e_suspect(self):
        return self.id in SUSPECTE_ID or self.nivel == "critical"
```

Fig.2.4 - Codul sursă

SUSPECTE_ID este o mulțime (set),deci verificarea self .id SUSPECTE_ID este rapidă. Condiția are două părți legate cu or: un eveniment este suspect dacă are un ID din listă sau davă nivelul lui este critical.

Proces și Eveniment au aceeași metodă e_suspect , dar fiecare cu regula ei. Aceasta este avantajul obiectelor: același nume de metodă , comportament potrivit fiecărui tip.

A4.Încărcarea din JSON

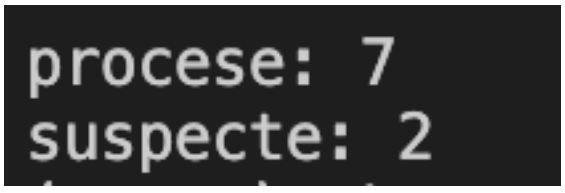


Fig.2.5- Rezultatul A4

III.

LUCRAREA B.

Clasa Inventar și raportul

B1.Inventar,o clasă-container

Inventar ține lista de obiecte Proces și are un constructor alternativ , din_json , care o umple.

```
@dataclass
class Inventar:
    procese: list

    @classmethod
    def din_json(cls, cale):
        with open(cale, encoding="utf-8") as f:
            return cls([Proces(**d) for d in json.load(f)])

    def __len__(self):
        return len(self.procese)

    def suspecte(self):
        return [p for p in self.procese if p.e_suspect()]

    def pe_user(self):
        grupe = {}
        for p in self.procese:
            grupe.setdefault(p.user, []).append(p.nume)
        return grupe
```

Fig.3.1- Codul sursă

```
procese: 7
```

Fig.3.2- Rezultatul B1

Explicație:

@classmethod primește clasa(cls) în loc de un obiect (self).De aceea Inventar.din_json(…) poate construi un inventar fără să existe deja unul.

cls([…]) este același lucru cu Inventar([..]),dar ramâne corect și dacă clasa este moștenită.

Metoda specială __len__ face ca len(inv) să funcționeze pe obiectul meu , ca la o listă.

B2. Metodele de interogare

Adaug pe Inventar două metode : suspecte() și pe_user().

```
procese: 7
  [3120] update.exe  C:\Users\ana\AppData\Local\Temp\update.exe
  [3890] svch0st.exe  C:\Users\ana\AppData\Roaming\svch0st.exe
{'SYSTEM': ['System', 'svchost.exe'], 'ana': ['explorer.exe', 'chrome.exe', 'update.exe',
'powershell.exe', 'svch0st.exe']}
(.venv) ekaterinalungul@MacBook-Air-Ekaterina Tehnici de Programare %
```

Fig.3.3 - Rezultatul B2

Explicație:

suspecte() nu repetă regula : întreabă fiecare proces prin p.e_suspect().Dacă regula se schimbă în Proces,Inventar rămâne corect.

pe_user() grupează numele procesol pe utilizator .grupe.setdefault(p.user, []),întoarce lista existentă a utilizatorului sau creează una goală dacă nu există , iar .append(p.nume) adaugă numele.

B3.Raportul JSON,din obiecte

asdict transformă un obiect înapoi în dicționar , ca să-l pot scrie în JSON.Funcția main() folosește argparse și produce un raport de alarme.

```
def main():
    import argparse
    p = argparse.ArgumentParser()
    p.add_argument("capturi")
    p.add_argument("--json", action="store_true")
    a = p.parse_args()

    inv = Inventar.din_json(a.capturi + "/procese.json")
    ev = evenimente_din_json(a.capturi + "/evenimente.json")
    alarma = [e for e in ev if e.e_suspect()]

    if a.json:
        raport = {
            "suspecte": [asdict(x) for x in inv.suspecte()],
            "alarma": [asdict(e) for e in alarma],
```

```
        }
        print(json.dumps(raport, indent=2, ensure_ascii=False))
    else:
        print(f"Procese: {len(inv)} | suspecte: {len(inv.suspecte())}")
        for x in inv.suspecte():
            print(f"  [{x.pid}] {x.nume}  ({x.user})  {x.cale}")
        print(f"Evenimente: {len(ev)} | alarma: {len(alarma)}")
        for e in alarma:
            print(f"  {e.id} [{e.nivel}] {e.cont}: {e.mesaj}")


if __name__ == "__main__":
    main()
```

Fig.3.4 - Codul sursă

```
Procese: 7 | suspecte: 2
  [3120] update.exe  (ana)  C:\Users\ana\AppData\Local\Temp\update.exe
  [3890] svch0st.exe  (ana)  C:\Users\ana\AppData\Roaming\svch0st.exe
Evenimente: 8 | alarma: 5
  4625 [warning] administrator: An account failed to log on
  4625 [warning] administrator: An account failed to log on
  4625 [warning] administrator: An account failed to log on
  7045 [warning] SYSTEM: A service was installed: WinUpdateSvc
  1102 [critical] administrator: The audit log was cleared
```

Fig.3.5 - Rezultatul B3

Rulare în modul JSON: python3 -m arsenal.windows.inventar arsenal/windows/start —json


```
  {
    "pid": 3120,
    "nume": "update.exe",
    "user": "ana",
    "cale": "C:\\Users\\ana\\AppData\\Local\\Temp\\update.exe"
  },
  {
    "pid": 3890,
    "nume": "svch0st.exe",
    "user": "ana",
    "cale": "C:\\Users\\ana\\AppData\\Roaming\\svch0st.exe"
  }
],
"alarma": [
  {
    "id": 4625,
    "nivel": "warning",
    "sursa": "Security",
    "cont": "administrator",
    "mesaj": "An account failed to log on"
  },
  {
    "id": 4625,
    "nivel": "warning",
    "sursa": "Security",
    "cont": "administrator",
    "mesaj": "An account failed to log on"
  },
  {
    "id": 4625,
    "nivel": "warning",
    "sursa": "Security",
    "cont": "administrator",
    "mesaj": "An account failed to log on"
  },
  {
    "id": 7045,
    "nivel": "warning",
    "sursa": "System",
    "cont": "SYSTEM",
    "mesaj": "A service was installed: WinUpdateSvc"
  },
  {
    "id": 1102,
    "nivel": "critical",
```


Fig.3.6 - Partea I JSON



Fig.3.7 - Partea II JSON

Explicație:

argparse definește argumentul obligatoriu capturi (folderul cu JSON-urile) și opțiunea --json. Cu action="store_true", a.json este True dacă opțiunea e prezentă și False altfel.

Lista alarma conține evenimentele care răspund True la e_suspect(): trei logon-uri eșuate (4625), un serviciu instalat (7045) și ștergerea jurnalului de audit (1102, nivel critical).

Succesiunea 4625 repetat, apoi 7045, apoi 1102 arată o posibilă secvență de atac: încercări de parolă, instalare de serviciu persistent și ștergerea urmelor.

B4.Testele

@dataclass oferă și __eq__: două obiecte se compară după câmpuri . Testele verifică metoda e_suspect.

Fișierul arsenal/windows/test_inventar.py:

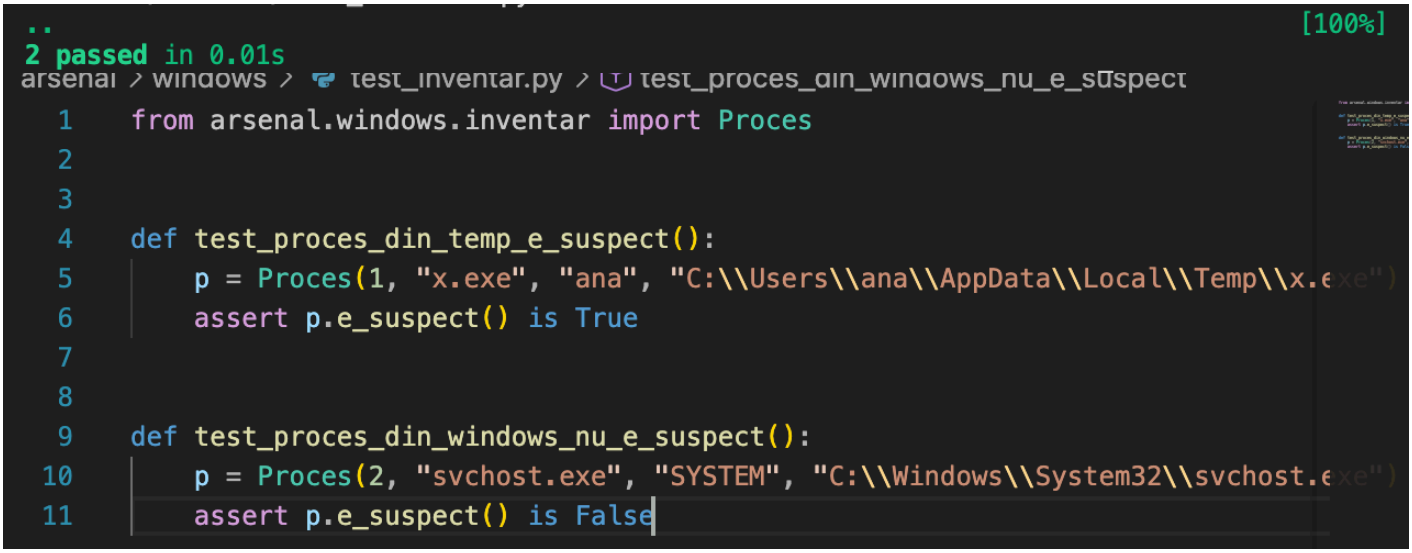


Fig.3.8 - Fișierul și codul sursă B4

Explicație:

Un test acoperă cazul pozitiv (proces din Temp este suspect), celălalt cazul negativ(svchost.exe din System32 nu este suspect).Împreună verifică ambele ramuri ale regulii.



Fig.3.9 - Rezultatul B4

CONCLUZII

Prin intermediul acestei lucrări de laborator nr.11, am modelat procese și evenimente Windows ca obiecte @dataclass, fiecare cu metoda e_suspect care răspunde singură. Am construit clasa_container Inventar, cu constructor alternativ din_json și metodele suspecte() și pe_user() , și un raport de alarme în format text sau JSON , comandat cu argparse.

Rezultatele coincid cu cele așteptate din fișă : 7 procese , 2 suspecte (update.exe și svch0st.exe),8 evenimente , 5 de alarmă , iar cele 2 teste trec cu pytest.

Lucrarea arată cum se transformă date brute dintr-o captură de sistem în obiecte cu comportament, ceea ce face detecția mai clară și mai ușor de extins.